🧠 Task Title: Unique Data Tracker and ID Generator System

🔹 Problem Statement

You are hired as a JavaScript developer for a data analytics startup. Your job is to build a **"Unique Data Tracker and ID Generator System"** — a mini system that manages user data dynamically, ensures uniqueness using **Sets**, maintains structured relationships with **Maps**, and creates custom **ID sequences using Generator Functions**.

The system should allow:

- Adding and managing unique users,
- Tracking data entries efficiently,
- Generating **unique incremental IDs** for each new user or record dynamically.

🎯 Objectives

By the end of this hands-on task, learners will:

1. **Understand real-world use cases** of Sets, Maps, and Generators.
2. Strengthen **logic-building** by integrating all three concepts into one functional system.
3. Practice **data handling, iteration, and uniqueness management** with JS core features.
4. Develop a **mini data management app** using only JavaScript — no backend or frameworks.

💻 Requirements

You must implement the following components:

1️⃣ Unique User Tracker (Using Sets)

- Create a Set to store unique user emails or usernames.
- A function addUser(name) should:
    - Add the user to the set.
    - Prevent duplicates.
    - Log a message if the user already exists.

Example:

```
addUser("alice@example.com"); // ✅ User added
addUser("alice@example.com"); // ⚠️ User already exists!
```

2️⃣ User Data Mapper (Using Maps)

- Use a Map to store user data in a structured way:
  Key → userEmail, Value → userData (object or score, etc.)
- Write a function updateUserData(email, data) that:
  - Adds new data if the user exists.
  - Logs an error if the user doesn't exist.

Example:

```
updateUserData("alice@example.com", { score: 90, level: 3 });
```

3️⃣ Unique ID Generator (Using Generator Functions)

- Create a generator function function* idGenerator() that yields sequential IDs (e.g., U001, U002, …).
- Each time a new user is added, assign them the next ID.

Example:

```
const gen = idGenerator();
console.log(gen.next().value); // U001
console.log(gen.next().value); // U002
```

Integrate this generator into your addUser() function so every new user automatically gets a unique ID.

4️⃣ Integrated System Logic

Combine everything:

- When a new user is added, generate an ID and add them to the Set and Map.

- When updating user data, verify that the user exists using the Set.
- Display all user data at the end in a clean, readable format.

⚡ **Extra Challenge (for advanced learners)**

Add these bonus features:

1. Create a **"search user"** function that looks up users in the Map by ID.
2. Add a **"remove user"** feature that deletes them from both the Set and Map.
3. Implement a **reset generator** option (e.g., if data is cleared, generator restarts from U001).
4. Bonus: Store multiple entries per user using nested Maps or Arrays.

🧩 **Expected Outcome**

By the end of this practical exercise, your JavaScript program should behave like a **mini data management system** that demonstrates strong understanding and integration of **Sets**, **Maps**, and **Generator Functions**.

1. Dynamic User Management with Sets

Your program should maintain a **Set** that holds a list of unique users — for example, their **email IDs** or **usernames**.
Since Sets automatically reject duplicates, this ensures no two users with the same name or email are added.

Example:

```
addUser("alice@example.com"); // ✅ User added
addUser("alice@example.com"); // ⚠️ User already exists!
```

✅ The program should **detect duplicates instantly** and display a message that the user already exists — no duplicate data should be stored.

2. Data Storage and Retrieval with Maps

Each user should have a structured record stored inside a **Map** —
where the **key** is the user's email (or unique ID), and the **value** is an **object** containing their data (like score, level, etc.).

Example:

```
updateUserData("alice@example.com", { score: 90, level: 3 });
updateUserData("bob@example.com", { score: 75, level: 2 });
```

✅ The Map helps in easily updating or retrieving each user's data later.
For example, you can look up a user's score in constant time.

3. Unique ID Assignment with Generator Functions

Every new user added should automatically get a **unique, sequential ID**, generated using a **Generator Function**.

This generator yields IDs like U001, U002, U003, etc., each time a new user joins.

Example:

```
const gen = idGenerator();
console.log(gen.next().value); // U001
console.log(gen.next().value); // U002
```

✅ Your system should integrate this generator into the addUser() function so that each user automatically gets a new unique ID assigned upon creation.

4. Integrated Logic and Output

All three components — Set, Map, and Generator — should work together as a **coordinated system**:

- The **Set** ensures uniqueness.
- The **Generator** creates unique IDs.
- The **Map** stores detailed user data.

After performing operations (like adding users or updating their data), your program should print a formatted output showing all current users and their details.

Sample Output :-

```
✅ New User Added: Alice (ID: U001)
✅ New User Added: Bob (ID: U002)
⚠️ User already exists: Alice
🟢 Data Updated: { email: 'alice@example.com', score: 90, level: 3 }
🔍 User Data:
ID: U001 | Email: alice@example.com | Score: 90 | Level: 3
ID: U002 | Email: bob@example.com | Score: 75 | Level: 2
```